浙江大学

本科实验报告

课程名称：	计算机组成与设计
姓　　名：	
学　　院：	信息与电子工程学院
系：	
专　　业：	电子科学与技术
学　　号：	
指导教师：	屈民军 唐奕

2024 年 10 月 13 日

专业：
姓名：
学号：
日期：
地点：教十一 400

浙江大学实验报告

课程名称：计算机组成与设计　指导老师：屈民军 唐奕　成绩：

实验名称：快速加法器的设计　实验类型：设计实验

一、实验目的

（1）掌握快速加法器的设计方法。
（2）熟悉流水线技术。
（3）掌握时序仿真的工作流程。

二、实验任务

1.采用“进位选择加法”技术设计 32 位加法器，并对设计进行功能仿真和时序仿真。
2.采用四级流水线技术设计 32 位加法器，并对设计进行功能仿真和时序仿真。

三、实验原理

1. 四位先行进位加法器的设计

两个加数分别为 $A_3A_2A_1A_0$ 和 $B_3B_2B_1B_0$，C_{-1} 为最低位进位。设两个辅助变量分别为 $G_3G_2G_1G_0$ 和 $P_3P_2P_1P_0$：$G_i = A_i \& B_i$、$P_i = A_i + B_i$。

一位全加器的逻辑表达式可转化为

$$\begin{cases} S_i = P_i\overline{G_i} \oplus C_{i-1} \\ C_i = G_i + P_iC_{i-1} \end{cases} \tag{5.12}$$

利用上述关系，一个四位加法器的进位计算就变转化为

$$\begin{cases} C_0 = G_0 + P_0C_{-1} \\ C_1 = G_1 + P_1C_0 = G_1 + P_1G_0 + P_1P_0C_{-1} \\ C_2 = G_2 + P_2C_1 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_{-1} \\ C_3 = G_3 + P_3C_2 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_{-1} \end{cases} \tag{5.13}$$

由式（5.13）可以看出，每一个进位的计算都直接依赖于整个加法器的最初输入，而不需要等待相邻低位的进位传递。理论上，每一个进位的计算都只需要三个门延迟时间，即产生 $G[i]$、$P[i]$ 的与门和或门，输入为 $G[i]$、$P[i]$、C_{-1} 的与门，以及最终的或门。同样道理，理论上最终结果 sum 的得到只需要四个门延迟时间。

实际上，当加数位数较大时，输入需要驱动的门数较多，其 VLSI 实现的输出时延增加很多，考虑到互连线的延时情况将会更加糟糕。因此，通常在芯片实现时先设计位数较少的超前进位加法器结构，而后以此为基本结构来构造位数较大的加法器。

2. 进位选择加法器结构

实际上，由超前进位加法器级联构成的多位加法器只是提高了进位传递的速度，其计算过程与行波进位加法器同样需要等待进位传递的完成。

借鉴并行计算的思想，人们提出了进位选择加法器结构，或者称为有条件的加法器结构（conditional sum adder），其算法的实质是增加硬件面积换取速度性能的提高。二进制加法的特点是进位或者为逻辑 1，或者为逻辑 0，二者必居其一。将进位链较长的加法器分为 M 块

分别进行加法计算，对除去最低位计算块外的 $M-1$ 块加法结构复制两份，其进位输入分别预定为逻辑 1 和逻辑 0。于是，M 块加法器可以同时并行进行各自的加法运算，然后根据各自相邻低位加法运算结果产生的进位输出，选择正确的加法结果输出。图5.6所示为 12 位进位选择加法器的逻辑结构图。12 位加法器划分为 3 块，最低一块（4 位）由 4 位超前进位加法器直接构成，后两块分别假设前一块的进位为 0 或 1 将两种结果都计算出来，再根据前级进位选择正确的和与进位。如果每一块加法结构内部都采用速度较快的超前进位加法器结构，那么进位选择加法器的计算时延为

$$t_{CSA} = t_{carry} + (M-2)t_{MUX} + t_{sum} \tag{5.14}$$

其中，t_{sum}、t_{carry} 分别为加法器的和与加法器的进位时延，t_{MUX} 为数据选择器的时延。

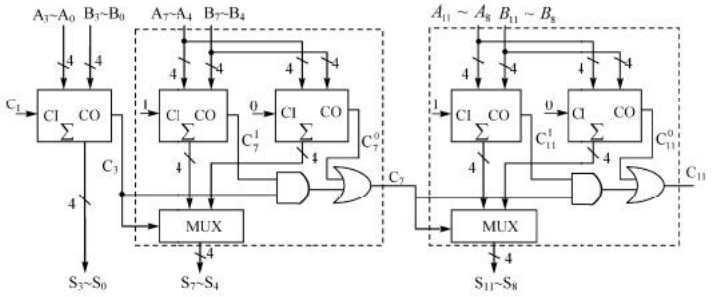


图 5.6　12 位进位选择加法器原理图

四、实验软件

Modelsim 、Vivado

五、实验步骤与过程

1. 4 位进位选择加法器的设计

（1）4 位进位选择加法器的 Verilog HDL 代码

```
module adder_4bits(a, b, ci, s, co);


    input[3:0] a, b;
    input ci;
    output[3:0] s;
    output co;
    wire[3:0] p, g;
    wire[2:0] c;


    assign g = a & b;
    assign p = a | b;
```

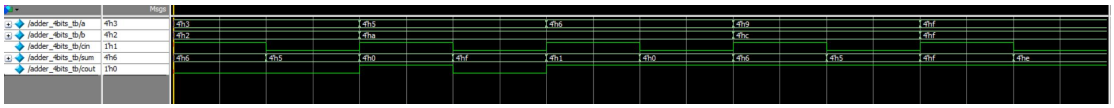
```
    //各级进位
    assign c[0] = g[0] | p[0]&ci;
    assign c[1] = g[1] | p[1]&c[0];
    assign c[2] = g[2] | p[2]&c[1];
    assign co = g[3] | p[3]&c[2];
```

```
    //求和
```

```
    assign s[0] = p[0]&(~g[0])&(~ci) | (~(p[0]&(~g[0])))&ci;
    assign s[1] = p[1]&(~g[1])&(~c[0]) | (~(p[1]&(~g[1])))&c[0];
    assign s[2] = p[2]&(~g[2])&(~c[1]) | (~(p[2]&(~g[2])))&c[1];
    assign s[3] = p[3]&(~g[3])&(~c[2]) | (~(p[3]&(~g[3])))&c[2];
```

```
endmodule
```

（2）ModelSim 仿真



经过检验，加和(sum)和进位(cout)均无误。

备注：sum, b, a 都调整为十进制无符号数(unsigned)格式显示，便于核验。cout,cin 因为只有 0,1 两个取值所以十六进制也不妨碍观察。

2. 32 位进位选择加法器的设计

（1）32 位进位选择加法器的 Verilog HDL 代码

将 32 位加法器分为 8 级，最低一级直接由四位超前进位加法器构成，剩下 7 级加法器都会同时计算进位 0,1 时的结果，并通过前一级的进位进行选择，作为最终该级加法器的输出。

该模块由顶层模块 adder_32bits、四位超前进位加法器 adder_4bits 和选择器 adder_mux 构成，代码如下：

```
    module adder_32bits (a, b, ci, s, co);
    input[31:0] a, b;
    input ci;
    output[31:0] s;
    output co;
    wire c1, c2, c3, c4, c5, c6;

    adder_4bits        adder0(.a(a[3:0]),      .b(b[3:0]),     .ci(ci), .s(s[3:0]), .co(c1));
    adder_mux adder1(.a(a[7:4]),      .b(b[7:4]),     .ci(c1), .s(s[7:4]), .co(c2));
    adder_mux  adder2(.a(a[11:8]),     .b(b[11:8]),    .ci(c2), .s(s[11:8]), .co(c3));
    adder_mux  adder3(.a(a[15:12]),    .b(b[15:12]),   .ci(c3), .s(s[15:12]), .co(c4));
    adder_mux  adder4(.a(a[19:16]),    .b(b[19:16]),   .ci(c4), .s(s[19:16]), .co(c5));
    adder_mux  adder5(.a(a[23:20]),    .b(b[23:20]),   .ci(c5), .s(s[23:20]), .co(c6));
    adder_mux  adder6(.a(a[27:24]),    .b(b[27:24]),   .ci(c6), .s(s[27:24]), .co(c7));
    adder_mux adder7(.a(a[31:28]),    .b(b[31:28]),   .ci(c7), .s(s[31:28]), .co(co));
endmodule
```

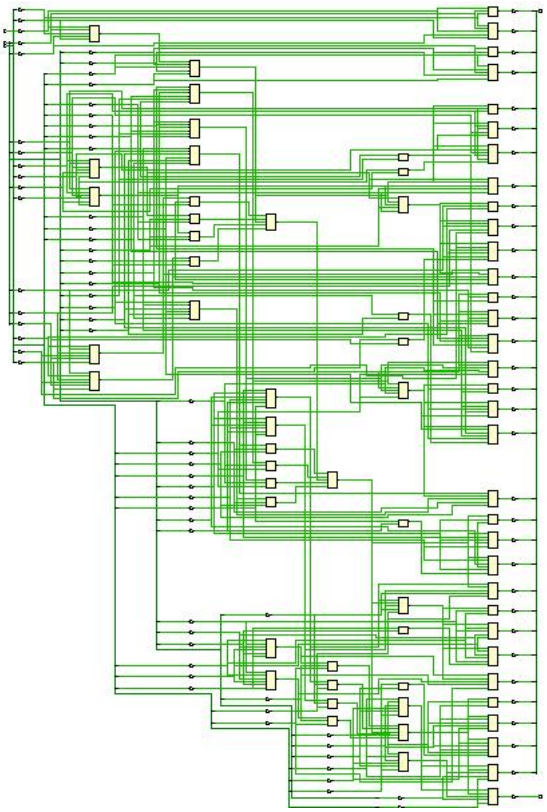
（2）ModelSim 仿真

经过检验，加和(s)和进位(co)均无误。

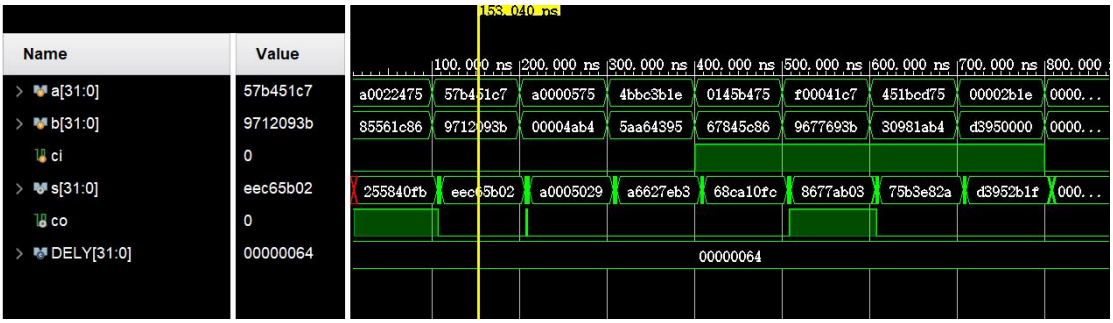
备注：s, b, a 都调整为十进制无符号数(unsigned)格式显示，便于核验。co,ci 因为只有 0,1 两个取值所以十六进制也不妨碍观察。

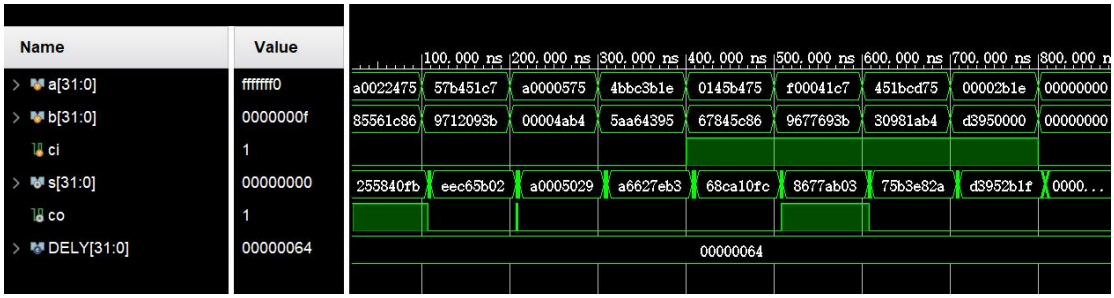
（3）进行时序仿真

综合原理图



仿真结果图:





存在传输延时和建立时间，但输出稳定后结果和 32 位进位选择加法器 ModelSim 仿真相同，是正确的。

特别需要注意，在计算中间，co 有一段为 1 的脉冲，但在该输入下，最后一级无论是否有前级进位都不会有进位的产生，所以认为是竞争冒险现象引起的。

六、实验中碰到的问题和解决方法

本次实验比较基础，在过程中并没有遇到太大的问题，只要按照实验书的思路，按步骤完成即可。其中主要的问题是 Verilog 语言有些遗忘，导致编译报错，有些细节层面经常报错，但是只要根据报错的提示，定位错误，检查语法，及时纠正，就没什么问题了/

七、思考题

1. 为什么要进行时序仿真？

时序仿真更加贴近电路实际的工作状态。如实验书中提及，32位选择进位加法器存在由进位实验、加和时延、数据选择器时延构成的计算时延。如果出现上一个加法计算的输出尚未稳定，我们就进入了下一拍，开始新一轮两数加法的计算，这样得到的结果是不可靠的。只有进行时序仿真，我们才更有理由判断，我们的稳定输出时间是否足够长，这可以指导我们实际中加法器的使用（即可以作为加法器正常工作时钟频率范围的依据）。另外，我们还能发现电路中是否存在竞争冒险，比如在本实验中，我们就观察到了co为0时出现的1型冒险。

2. 采用流水线技术有什么优缺点？

优点：将电路切分为多个模块，时钟频率只要满足多个模块之间最慢的模块的所需要的时间即可，而不是串行的延迟时间的叠加，从而可以提高时钟频率，

进而减少执行时间。同时也增加了数据的吞吐率和硬件的利用率。

缺点：需要考虑数据冒险、结构冒险、控制冒险等依赖问题，电路设计的难度和复杂度更高，而且生产成本、功耗更高。